Performance Comparison of RSA and ECDSA Digital Signatures

Lucas Bixby | CS 410 | 10/26/2026

1. Introduction

The goal in this exercise was to compare RSA and ECDSA digital signatures by measuring how long signing and verification take across two variables: key strength (RSA 2048 and 4096-bit; ECDSA prime256v1 and secp384r1) and payload size (1 MB, 10 MB, and 100 MB). Each operation was run through the OpenSSL command-line tool from a Python script.

2. Methodology

Environment. Benchmarks ran on a MacBook Air (Apple M2, 2022; 8 GB memory; macOS Sequoia 15.7.3) with Python 3.9.6 and OpenSSL 3.6.3. The machine was otherwise idle during testing and plugged in.

Payload files. Three random binary files of exactly 1 MB, 10 MB, and 100 MB (file_1MB.bin, file_10MB.bin, file_100MB.bin) were written from `os.urandom`, equivalent to `dd if=/dev/urandom bs=1M count=N`. The same files were used for every algorithm.

Keys. Alice generated an RSA key pair at 2048 and 4096 bits (`openssl genrsa`) and an ECDSA key pair on prime256v1 and secp384r1 (`openssl ecparam -genkey`). Each public key was extracted with `-pubout` and copied to Bob's folder (receiver/rsa_workspace or receiver/ecdsa_workspace).

Signing and verification. For each key and file, Alice signed with `openssl dgst -sha256 -sign` and copied the file and signature to Bob's folder. Bob verified with `openssl dgst -sha256 -verify` using Alice's public key; every run returned "Verified OK" before it was timed.

Timing. Elapsed time was measured with `time.perf_counter()` around each OpenSSL call. Each measurement had one unrecorded warm-up run, then 10 recorded runs; results are the mean and standard deviation (SD). Each timing includes the cost of launching the OpenSSL process (about 7.01 ms, measured with `openssl version`). Raw primitive speed was also measured with `openssl speed rsa2048 rsa4096 ecdsap256 ecdsap384`, reported in Table 3.

Scripts. `sender/rsa_workspace/alice.py` and `sender/ecdsa_workspace/alice.py`. Results are saved to results/*.csv and charts to charts/. The full source code is linked in the Appendix.

3. Results

3.1 RSA

Table 1. RSA signature generation and verification time (mean and SD over 10 runs). Values from results/rsa_results.csv.

RSA Key Size	File Size	Sign Time (ms)	Sign SD (ms)	Verify Time (ms)	Verify SD (ms)	Sig Size (Bytes)
2048-bit	1 MB	11.34	0.16	9.72	0.62	256
2048-bit	10 MB	20.64	0.83	17.97	0.35	256
2048-bit	100 MB	112.80	2.80	109.87	2.90	256
4096-bit	1 MB	17.42	0.23	9.32	0.15	512
4096-bit	10 MB	26.08	0.50	17.99	0.77	512
4096-bit	100 MB	118.74	2.02	113.38	13.77	512

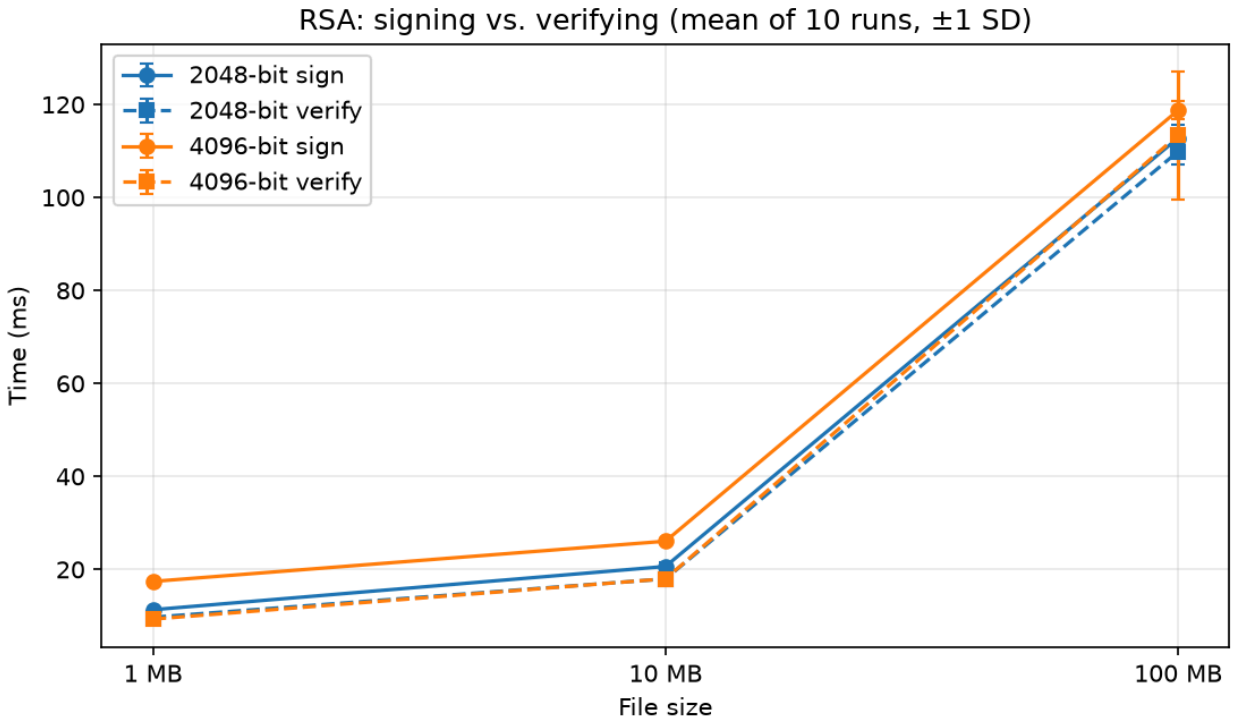


Figure 1. RSA signing vs. verifying time by file size. Solid lines are signing, dashed lines are verifying; error bars are ±1 SD over 10 runs.

3.2 ECDSA

Table 2. ECDSA signature generation and verification time (mean and SD over 10 runs). Values from results/ecdsa_results.csv.

ECDSA Curve	File Size	Sign Time (ms)	Sign SD (ms)	Verify Time (ms)	Verify SD (ms)	Sig Size (Bytes)
prime256v1 (256b)	1 MB	10.20	0.35	9.70	0.69	70
prime256v1 (256b)	10 MB	19.47	1.10	21.06	6.94	72
prime256v1 (256b)	100 MB	114.54	6.97	114.10	2.98	71
secp384r1 (384b)	1 MB	10.34	0.46	9.78	0.15	102
secp384r1 (384b)	10 MB	18.70	0.52	18.31	1.00	103
secp384r1 (384b)	100 MB	112.15	3.15	106.11	3.73	103

Note: the handout's Table 2 lists secp384r1 at 100 MB twice; the second row was measured at 10 MB to match the other algorithms.

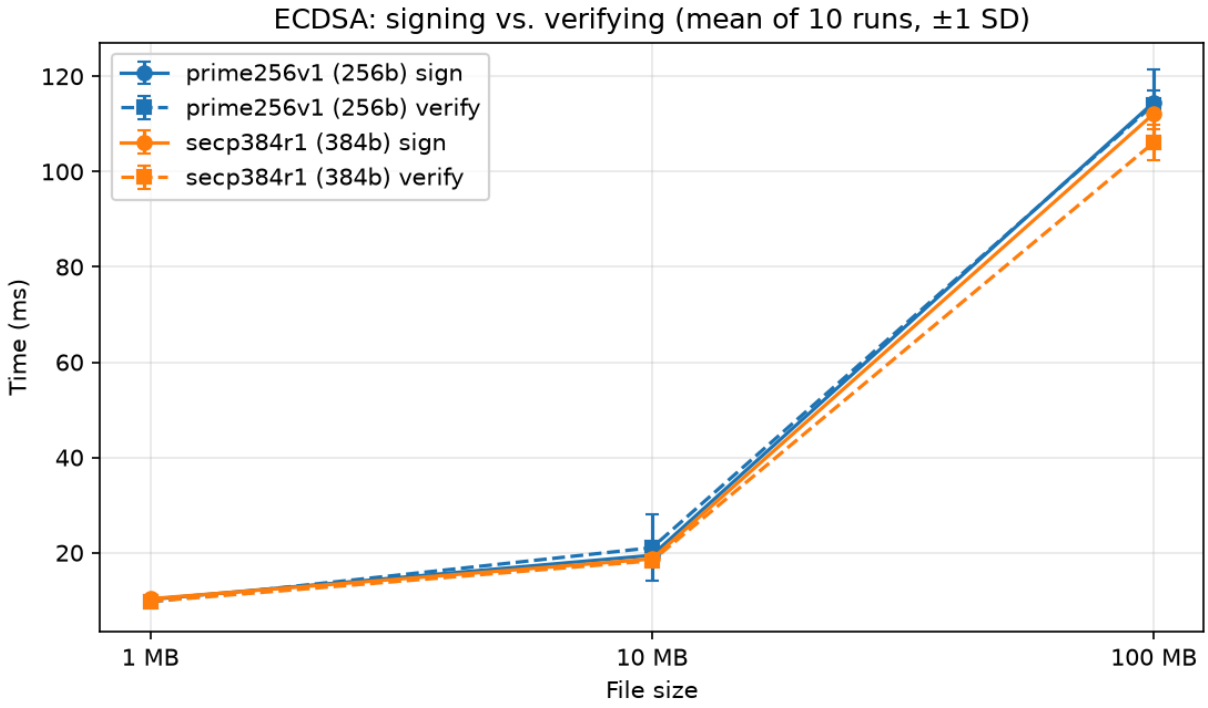


Figure 2. ECDSA signing vs. verifying time by file size. Solid lines are signing, dashed lines are verifying; error bars are ±1 SD over 10 runs.

3.3 RSA vs. ECDSA

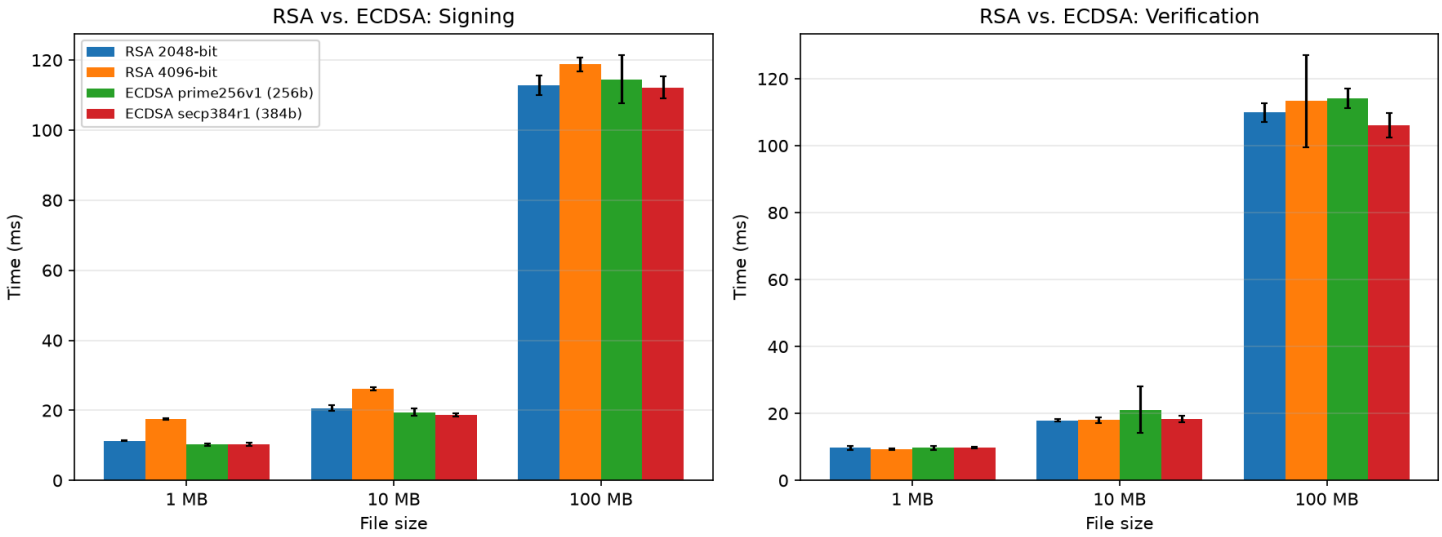


Figure 3. Signing (left) and verification (right) time for all four algorithms at each file size. Error bars are ±1 SD.

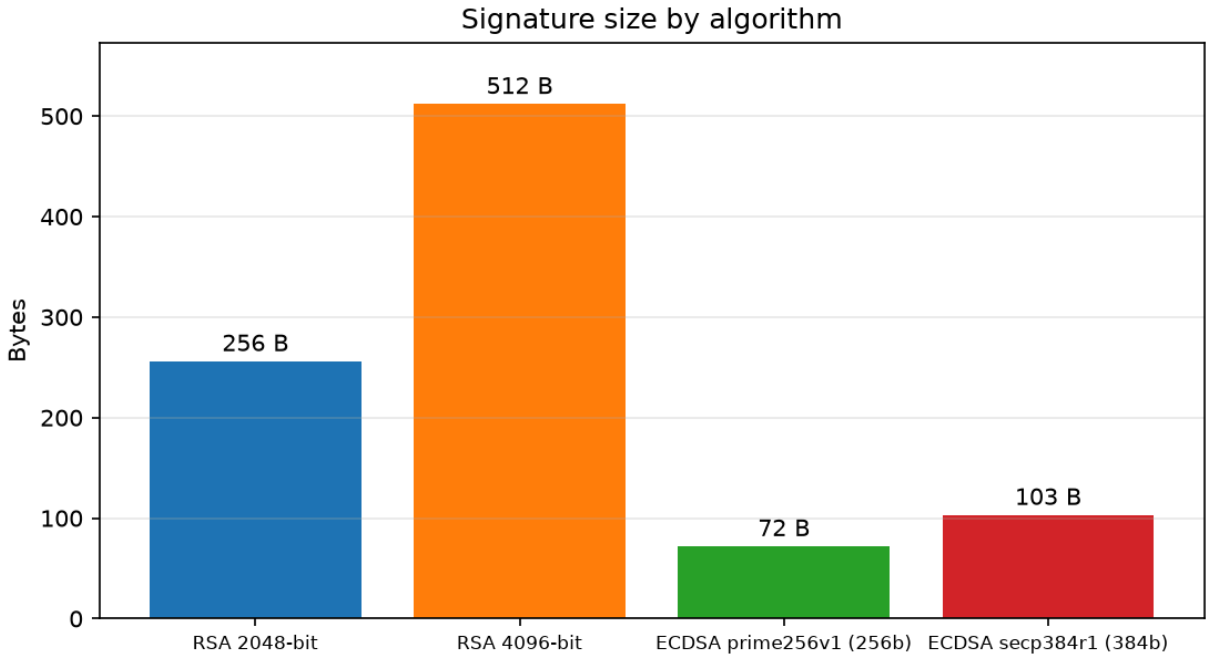


Figure 4. Signature size by algorithm. ECDSA sizes vary by 1 to 2 bytes between runs because the signature is DER-encoded; the largest observed size is shown.

3.4 Raw primitive speed (`openssl speed`)

Table 3. Signing and verification rate without hashing or process start-up, from `openssl speed rsa2048 rsa4096 ecdsap256 ecdsap384`.

Algorithm	Sign (ops/s)	Verify (ops/s)	Verify ÷ Sign
RSA 2048	1,080.0	42,862.4	39.7×
RSA 4096	170.6	11,724.1	68.7×
ECDSA P-256	34,206.7	11,667.2	0.34×
ECDSA P-384	4,296.1	2,026.2	0.47×

4. Comparative Analysis

RSA key size: signing vs. verification. Moving from 2048 to 4096 bits raised signing time at 1 MB from 11.34 ms to 17.42 ms, while verification stayed near 9.5 ms for both (9.72 and 9.32 ms; Figure 1). Verification uses the 17-bit public exponent e = 65537, so it takes the same 16 squarings at any key size, while signing uses a private exponent d as long as the key itself and slows down roughly with the cube of the key size. This shows in Table 3: doubling the key slowed signing by 6.3× (1,080.0 to 170.6 ops/s) but verification by only 3.7×, widening the Verify ÷ Sign ratio from 39.7× to 68.7×.

ECDSA signing vs. verification. At every file size, signing and verification took about the same time for both curves (10.20 vs. 9.70 ms for P-256 and 10.34 vs. 9.78 ms for P-384 at 1 MB; Figure 2), because hashing and process start-up hide the difference. The raw speed in Table 3 shows ECDSA is asymmetric in the opposite direction from RSA: P-256 signed 2.9× faster than it verified (34,206.7 vs. 11,667.2 ops/s) and P-384 2.1× faster (4,296.1 vs. 2,026.2 ops/s), because signing needs one elliptic-curve point multiplication and verification needs two.

Effect of file size: hashing vs. cryptographic math. Growing the file from 1 MB to 100 MB raised every algorithm from 10.20–17.42 ms to 106.11–118.74 ms, and all four converged at 100 MB (Figure 3), so SHA-256 hashing dominates for large files. The signature math shows only as a small fixed offset, visible mainly at 1 MB, where about 7.01 ms of each timing is OpenSSL process start-up rather than hashing or signing.

Signature sizes. RSA signatures equal the key size: 256 B (2048-bit) and 512 B (4096-bit), regardless of file size. ECDSA signatures were 70–72 B (P-256) and 102–103 B (P-384), about 3.6–5× smaller (Figure 4); the 1–2 byte variation comes from DER encoding.

High-throughput server choice. For a server signing millions of TLS handshakes, ECDSA P-256 is the better choice because it signed about 32× faster than RSA-2048 in openssl speed (34,206.7 vs. 1,080.0 signatures/s). For clients or IoT nodes that mostly verify, RSA-2048 is faster (42,862.4 vs. 11,667.2 verifications/s for P-256, about 3.7×), but ECDSA's much smaller signatures (72 B vs. 256 B; Tables 1–2) and keys save bandwidth and storage, which often makes ECDSA the better overall fit for constrained devices too.

Variability. SD stayed within about 7% of the mean for every measurement except two: RSA-4096 verification at 100 MB (113.38 ± 13.77 ms, about 12%) and P-256 verification at 10 MB (21.06 ± 6.94 ms, about 33%). Those outliers likely come from background activity or disk caching during individual runs, so differences between algorithms smaller than the SD should not be treated as meaningful.

5. Conclusion

On this machine, all four algorithms took about the same time for large payloads (106–119 ms at 100 MB), because SHA-256 hashing outweighed the signature math. RSA signing slowed sharply as the key grew while verification stayed cheap, whereas ECDSA signed 2–3× faster than it verified, and its signatures were 3.6–5× smaller (72 B vs. 256 B; 103 B vs. 512 B). For signing-heavy systems such as TLS servers, ECDSA P-256 is the better choice at about 32× RSA-2048's signing rate, while RSA remains a reasonable option for verification-heavy clients, where it is verified about 3.7× faster than P-256.

Appendix

Source code. [GitHub link]